

系统开发工具基础

肖扬

计算机科学与技术

学号：25020007140

2026 年 9 月 6 日

目录

1	实验环境说明	1
2	课堂学习部分	2
2.1	第 5 题控制一个可清理的后台任务	2
2.1.1	实验脚本及原理	2
2.1.2	实验操作过程	2
2.2	第 6 题语义重构与本地开发反馈	7
2.2.1	实验操作过程	7
2.3	第 7 题用调试器定位归并排序缺陷	15
2.3.1	实验操作过程	15
2.4	第 8 题先测量再优化慢速词频程序	24
2.4.1	实验操作过程	24
2.4.2	中位数与加速比计算	28
3	课后练习部分	30
3.1	课后第 1 题	30
3.1.1	实验操作过程	30
3.2	课后第 2 题	31
3.2.1	实验操作过程	31
3.3	课后第 3 题	33
3.3.1	实验操作过程	34
3.4	课后第 4 题	37
3.4.1	实验操作过程	37
3.5	课后第 5 题	40

3.5.1	实验操作过程	40
3.6	课后第 6 题	41
3.6.1	实验操作过程	41
3.7	课后第 7 题	42
3.8	课后第 8 题	45
3.9	课后第 9 题	46
3.10	课后第 10 题	48
4	解题感悟	51
4.1	课上练习	51
4.2	课后练习	51
5	上传 GitHub	53
5.1	编写并更新.gitignore	53
5.2	GitHub 仓库链接	54
5.3	commit 提交记录	54

Chapter 1

实验环境说明

1. 系统版本：Ubuntu 22.04.1 LTS，用户名 xy_ai，WSL2 架构
2. 编译器：gcc 11.4.0、GNU Make 4.3（都在 /usr/bin）
3. shell：bash 5.1.16（x86_64-pc-linux-gnu）
4. git：2.34.1（Ubuntu 22.04 官方源版本，系统级）
5. LaTeX：TeX Live 2022/dev（Debian 版），含 xelatex、pdflatex、latexmk
6. 所有任务均在 base 这个环境中进行，没有使用其他环境

Chapter 2

课堂学习部分

2.1 第 5 题控制一个可清理的后台任务

2.1.1 实验脚本及原理

题目中给出的脚本为 `worker.sh`，内容如下：

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

第一行使用当前环境中的 Bash 解释脚本。`trap` 为 `TERM` 和 `INT` 信号设置处理动作：脚本收到其中任一信号时，先向 `cleanup.log` 追加 `CLEAN_EXIT`，然后以状态码 0 退出。变量 `n` 从 0 开始，每轮循环输出当前值、加 1，并等待 1 秒。`while true` 使脚本持续运行，以便观察作业状态和信号处理过程。

2.1.2 实验操作过程

1. 创建实验目录并确认路径

在 WSL2 的 Ubuntu 终端中输入：

```
mkdir q05
cd q05
pwd
```

`mkdir q05` 用于建立独立的实验目录，使本题的脚本和日志集中保存；`cd q05` 进入该目录；`pwd` 用于确认实际工作位置。终端输出为：

```
/home/xy_ai/q05
```

2. 创建并检查 worker.sh

```
nano worker.sh
```

在 `nano` 中输入题目要求的脚本，保存并退出。随后执行：

```
cat worker.sh
```

3. 增加执行权限并验证

```
chmod +x worker.sh  
ls -l worker.sh
```

使用 `chmod +x` 增加执行权限。`ls -l` 显示：

```
-rwxr-xr-x 1 xy_ai xy_ai 140 Sep  6 22:15 worker.sh
```

4. 在前台启动脚本并分别重定向标准输出和标准错误

```
./worker.sh > stdout.log 2> stderr.log
```

> 把标准输出，也就是文件描述符 1，写入 `stdout.log`；2> 把标准错误写入 `stderr.log`。把两种输出分开保存，可以分别检查正常数据和错误信息。

命令末尾没有 `&`，因此脚本最初应该在前台运行。

5. 使用 Ctrl-Z 暂停前台任务

等待脚本运行一段时间后，我按下 `Ctrl-Z`。终端显示：

```
[1]+  Stopped      ./worker.sh > stdout.log 2> stderr.log
```

`Ctrl-Z` 会向当前前台进程组发送暂停信号，使进程停止执行但不退出。随后输入：

```
jobs
```

作业状态仍为 `Stopped`。其中 `[1]` 是 `Bash` 分配的作业号，说明该任务仍由当前 `Shell` 管理，只是暂时没有继续运行。

6. 将暂停的任务放到后台继续运行

```
bg
jobs
```

bg 恢复最近暂停的任务，并使它在后台运行。再次查看 jobs 时，状态已经由 Stopped 变为 Running，命令末尾还显示 &。这说明脚本已经恢复计数，同时终端可以继续输入其他命令。

```
(base) xy_ai@Lenovo:~$ mkdir q05
(base) xy_ai@Lenovo:~$ cd q05
(base) xy_ai@Lenovo:~/q05$ pwd
/home/xy_ai/q05
(base) xy_ai@Lenovo:~/q05$ nano worker.sh
(base) xy_ai@Lenovo:~/q05$ cat worker.sh
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
(base) xy_ai@Lenovo:~/q05$ chmod +x worker.sh
(base) xy_ai@Lenovo:~/q05$ ls -l worker.sh
-rwxr-xr-x 1 xy_ai xy_ai 140 Sep  6 22:15 worker.sh
(base) xy_ai@Lenovo:~/q05$ ./worker.sh > stdout.log 2> stderr.log

^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
(base) xy_ai@Lenovo:~/q05$ jobs
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
(base) xy_ai@Lenovo:~/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
(base) xy_ai@Lenovo:~/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
(base) xy_ai@Lenovo:~/q05$ tail stdout.log
48
49
50
51
52
53
54
```

图 2.1: 创建目录、编写脚本、增加执行权限、暂停任务并在后台恢复

7. 使用 tail 验证后台任务仍在输出

```
tail stdout.log
```

tail 默认显示文件最后 10 行。第一次截图中可以看到 48 至 54；稍后再次执行时，可以看到 77 至 86。末尾数值继续增加，证明脚本已经在后台恢复运行，并持续向 stdout.log 写入数据。

8. 获取并保存后台任务的 PID

```
jobs -p
```

`jobs -p` 只输出当前 Shell 所管理作业的进程号。本次实验得到：

```
60
```

```
pid=$(jobs -p)
echo "$pid"
```

`$(jobs -p)` 是命令替换，Bash 会把命令输出保存到变量 `pid` 中。`echo` 再次输出 60，说明变量赋值正确。

```
(base) xy_ai@Lenovo:~/q05$ tail stdout.log
77
78
79
80
81
82
83
84
85
86
(base) xy_ai@Lenovo:~/q05$ jobs -p
60
(base) xy_ai@Lenovo:~/q05$ pid=$(jobs -p)
(base) xy_ai@Lenovo:~/q05$ echo "$pid"
60
(base) xy_ai@Lenovo:~/q05$ kill -TERM "$pid"
(base) xy_ai@Lenovo:~/q05$ wait "$pid"
[1]+  Done                  ./worker.sh > stdout.log 2> stderr.log
(base) xy_ai@Lenovo:~/q05$ cat cleanup.log
CLEAN_EXIT
(base) xy_ai@Lenovo:~/q05$ jobs
(base) xy_ai@Lenovo:~/q05$ cat stdout.log
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

图 2.2: 取得 PID 60、发送 SIGTERM、验证清理记录并查看标准输出

9. 发送 SIGTERM 并等待任务结束


```
kill -TERM "$pid"
wait "$pid"
```

`kill -TERM` 向 PID 60 发送 `SIGTERM`。这是一种可被进程捕获的终止请求，因此脚本中的 `trap` 有机会先执行清理动作。这里不能用无法捕获的 `SIGKILL`，否则脚本来不及写入清理日志。

`wait` 等待指定子进程结束，并让当前 Shell 回收其退出状态。终端显示：

```
[1]+  Done      ./worker.sh > stdout.log 2> stderr.log
```

`Done` 表示后台作业已经结束。

10. 验证清理日志和作业列表

```
cat cleanup.log
jobs
```

`cat cleanup.log` 输出：

```
CLEAN_EXIT
```

这证明 `SIGTERM` 已经送达，脚本中的 `trap` 被成功触发，并在退出前完成了写入清理标记的操作。随后执行 `jobs` 没有任何输出，说明当前 Shell 中已经没有运行或暂停的后台作业。

11. 检查 `stdout.log` 和 `stderr.log`

```
cat stdout.log
cat stderr.log
```

`stdout.log` 中保存了从 0 开始的连续递增整数。截图显示文件开头为 0、1、2、3 等，末尾可见 120 至 128，说明脚本的标准输出被正确保存，且后台恢复后继续写入。执行 `cat stderr.log` 时终端没有显示任何内容，说明本次运行没有产生标准错误。

```
120
121
122
123
124
125
126
127
128
(base) xy_ai@Lenovo:~/q05$ cat stderr.log
(base) xy_ai@Lenovo:~/q05$ |
```

图 2.3: 标准输出末尾为 128，标准错误文件为空

2.2 第 6 题语义重构与本地开发反馈

2.2.1 实验操作过程

1. 使用 VS Code 创建三个 Python 文件

我在 VS Code 中创建 `math_utils.py`、`app.py` 和 `test_math_utils.py`。内容如下。

`math_utils.py`:

```
def total_price(price: float, count: int) -> float:
    return price * count
```

该函数接收单价和数量，返回二者的乘积。

`app.py`:

```
from math_utils import total_price

print(total_price(12.5, 4))
```

该文件从 `math_utils` 导入函数，以单价 12.5、数量 4 调用函数并打印结果。

`test_math_utils.py`:

```
from math_utils import total_price

def test_total_price():
```

```
assert total_price(12.5, 4) == 50.0
```

测试使用 `assert` 检查函数是否返回预期的 50.0。测试文件名和测试函数名都是符合 Pytest 的自动发现规则。

2. 运行重构前的程序与测试

```
python app.py  
pytest -q
```

程序输出 50.0，Pytest 显示 1 passed。



图 2.4

3. 使用语言服务器查看函数信息并跳转到定义

在 `app.py` 中把鼠标停在 `total_price` 上，编辑器显示了函数签名、参数类型和返回类型：

```
def total_price(price: float, count: int) -> float
```

随后右键该函数并选择“转到定义”，VS Code 自动打开 `math_utils.py`，并定位到函数定义。这证明语言服务器已经理解 `app.py` 中的调用来自 `math_utils.py`。

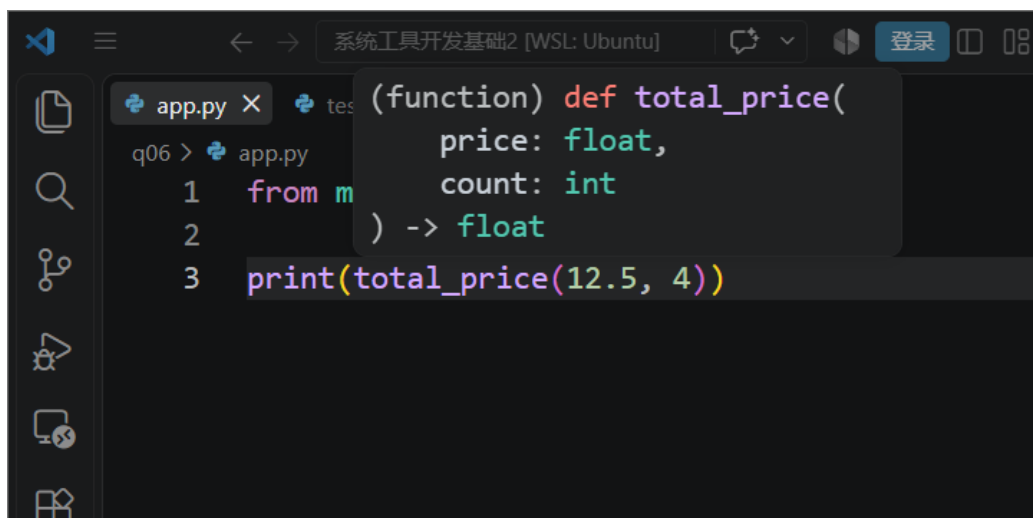


图 2.5

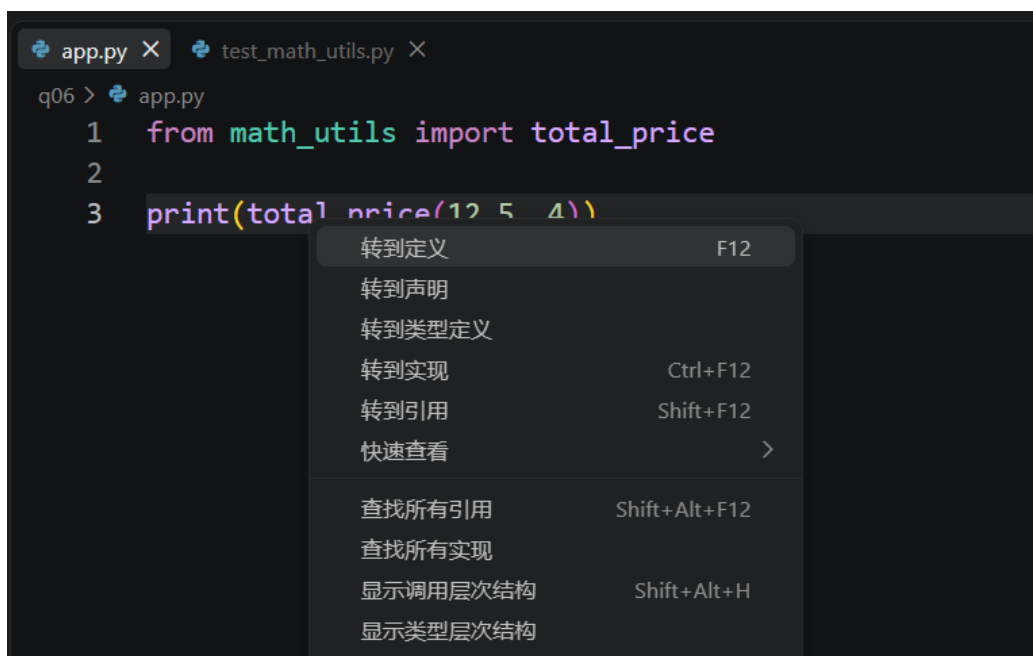


图 2.6

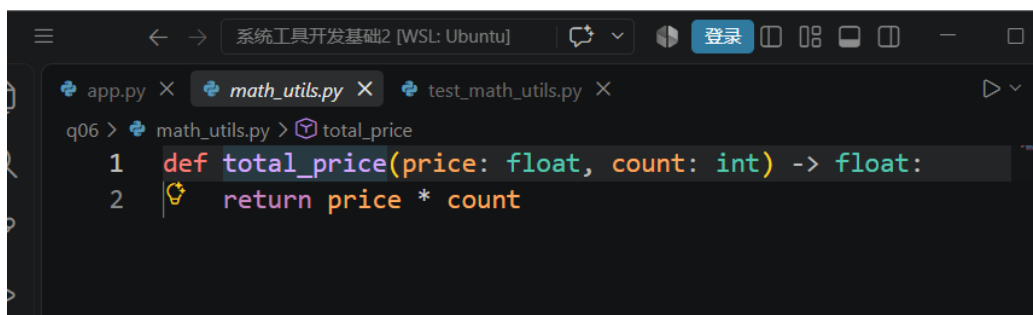


图 2.7

4. 查找 total_price 的全部引用

在函数定义处右键选择“查找所有引用”。VS Code 的引用面板显示“3 个文件中 5 个结果”，涉及 `math_utils.py`、`app.py` 和 `test_math_utils.py`。这些结果包括函数定义、两处导入以及两处调用。手动修改定义，另外两个文件仍导入旧名称，程序会发生导入错误。

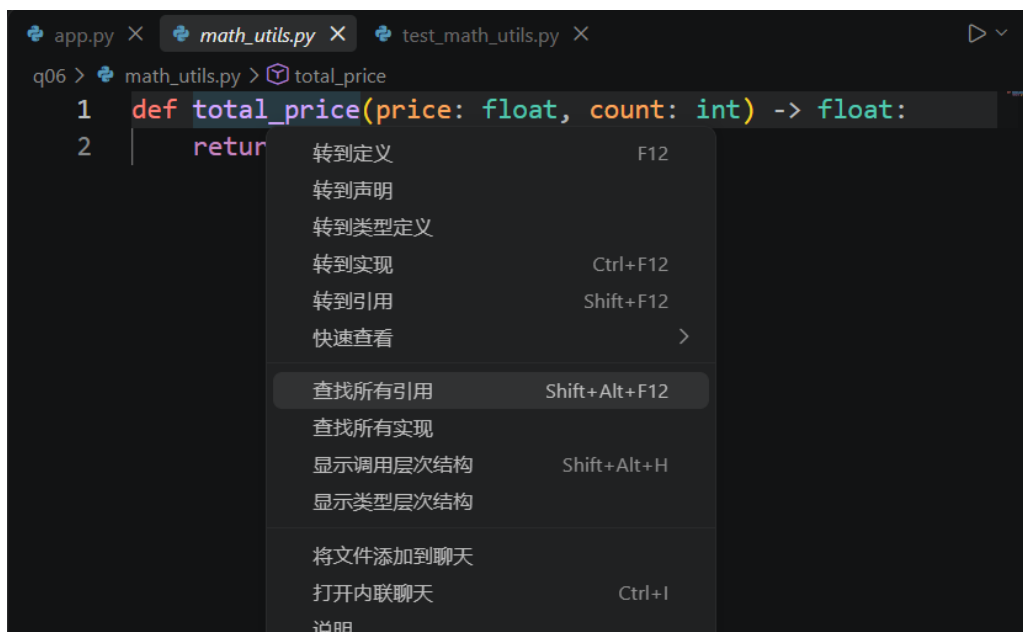


图 2.8

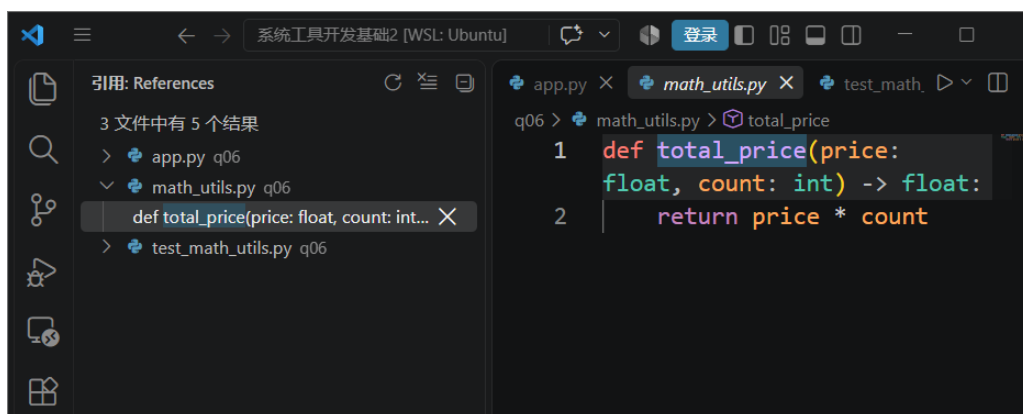


图 2.9

5. 使用“重命名符号”完成语义重构

将光标放在 `math_utils.py` 的函数名 `total_price` 上，按 F2 调出“重命名符号”，输入：

```
calculate_total
```

语言服务器同步修改了函数定义、`app.py` 中的导入与调用,以及 `test_math_utils.py` 中的导入与调用。最终三个文件的有效代码如下:

`math_utils.py`:

```
def calculate_total(price: float, count: int) -> float:
    return price * count
```

`app.py`:

```
from math_utils import calculate_total

print(calculate_total(12.5, 4))
```

`test_math_utils.py`:

```
from math_utils import calculate_total

def test_total_price():
    assert calculate_total(12.5, 4) == 50.0
```

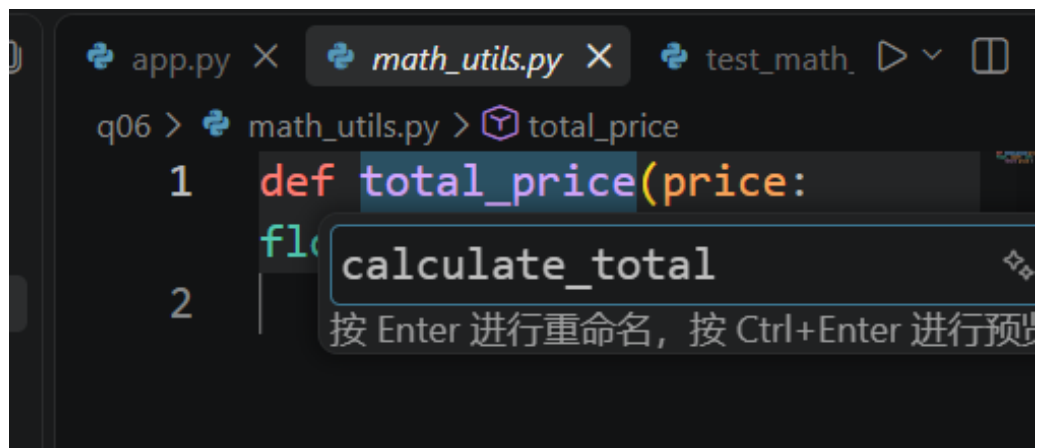


图 2.10: 使用重命名符号将 `total_price` 改为 `calculate_total`

```
app.py × math_utils.py × test_math_utils.py ×
q06 > app.py
1 from math_utils import calculate_total
2
3 print(calculate_total(12.5, 4))
```

图 2.11

```
app.py × math_utils.py × test_math_utils.py ×
q06 > test_math_utils.py > test_total_price
1 from math_utils import calculate_total
2
3
4 def test_total_price():
5     assert calculate_total(12.5, 4) == 50.0
```

图 2.12

```
(base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ python app.py
50.0
(base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ pytest -q
.
1 passed in 0.01s
(base) xy_ai@Lenovo:~/系统工具开发基础2/q06$
```

图 2.13: 程序仍输出 50.0，测试仍然通过

6. 使用 Ruff 发现未使用的 import os

按题目要求，我先在 app.py 的第一行临时加入：

```
import os
```

因为后面的代码没有使用 os，所以这是一个没有作用的导入。保存后执行：

```
ruff check .
```

Ruff 报告 F401, 指出 `os` imported but unused, 位置是 `app.py:1:8`, 并提示该问题可以使用 `--fix` 自动修复。

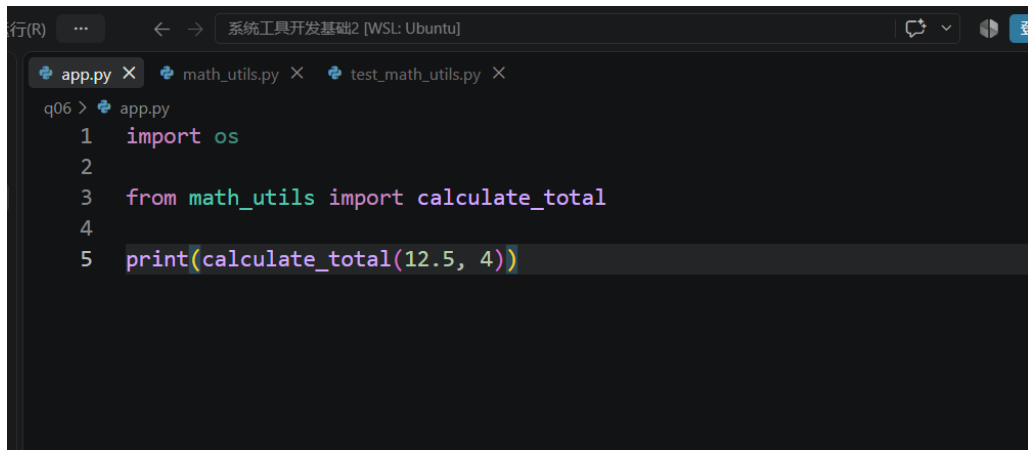


图 2.14

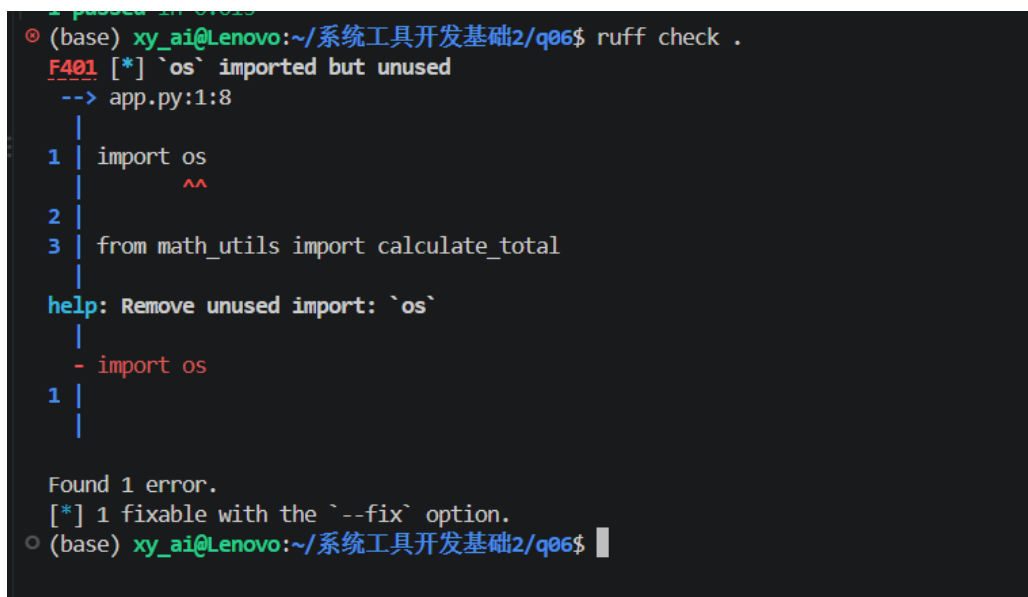


图 2.15

7. 使用 Ruff 自动修复并进行最终验证

```
ruff check . --fix
```

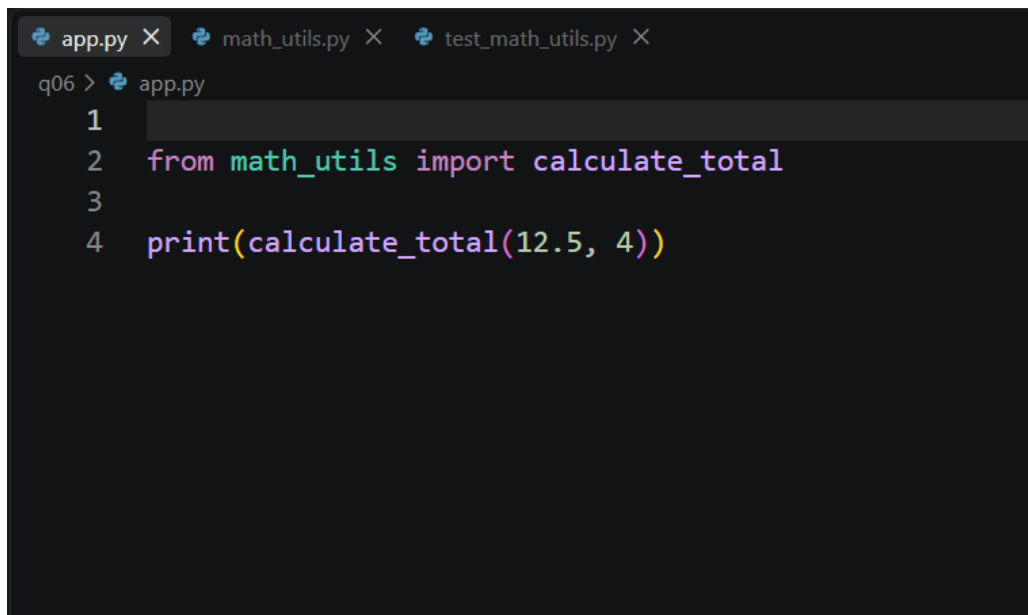
终端显示发现 1 个错误、已修复 1 个错误、剩余 0 个错误。Ruff 自动删除了未使用的 `import os`。


```
ruff check .
pytest -q
python app.py
```

Ruff 输出 All checks passed!, Pytest 输出 1 passed in 0.01s, 程序仍然输出 50.0。

```
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ ruff check . --fix
  Found 1 error (1 fixed, 0 remaining).
○ (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$
```

图 2.16: Ruff 自动修复一个错误，剩余错误为零



```
app.py X math_utils.py X test_math_utils.py X
q06 > app.py
1
2 from math_utils import calculate_total
3
4 print(calculate_total(12.5, 4))
```

图 2.17: 自动修复删除 import os

```
Found 1 error (1 fixed, 0 remaining).
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ ruff check .
  All checks passed!
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ pytest -q
.
1 passed in 0.01s
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$ python app.py
50.0
○ (base) xy_ai@Lenovo:~/系统工具开发基础2/q06$
```

图 2.18

2.3 第 7 题用调试器定位归并排序缺陷

2.3.1 实验操作过程

1. 创建 q07 并保存缺陷代码

```
cd "/home/xy_ai/系统工具开发基础2"
mkdir -p q07
cd q07
```

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[i])
            j += 1
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

归并排序先递归地把列表分成更小的部分，再由 `merge` 合并两个已排序的子列表。变量 `i` 应当表示左侧列表当前位置，`j` 应当表示右侧列表当前位置。缺陷代码在进入 `else` 分支时却使用了 `right[i]`。

```
merge_sort.py X
q07 > merge_sort.py > ...
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i])
7             i += 1
8         else:
9             result.append(right[j])
10            j += 1
11    return result + left[i:] + right[j:]
12
13
14 def merge_sort(a):
15     if len(a) <= 1:
16         return a
17     m = len(a) // 2
18     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
19
20
21 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

问题 输出 调试控制台 终端 端口 2
+ v bash - q07
● (base) xy_ai@Lenovo:~/系统工具开发基础2$ mkdir -p q07
● (base) xy_ai@Lenovo:~/系统工具开发基础2$ cd q07
○ (base) xy_ai@Lenovo:~/系统工具开发基础2/q07$
```

图 2.19

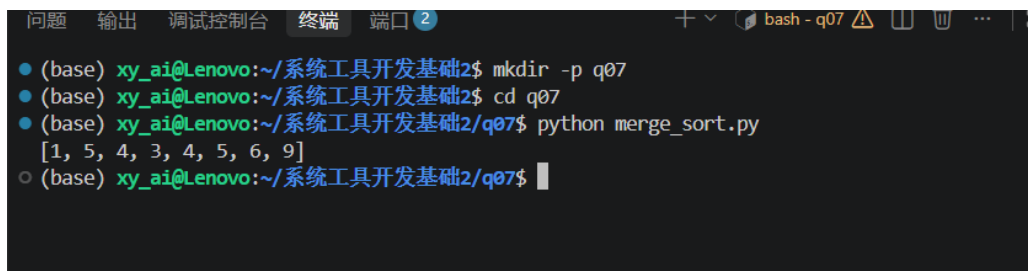
2. 运行缺陷代码并确认错误结果

```
python merge_sort.py
```

终端输出:

```
[1, 5, 4, 3, 4, 5, 6, 9]
```

正确结果应为 [1, 1, 2, 3, 4, 5, 6, 9]。实际输出不仅没有保持升序，还丢失了一个 1，多出了一个 5。这证明程序可以运行，但是逻辑存在错误。



```
(base) xy_ai@Lenovo:~/系统工具开发基础2$ mkdir -p q07
(base) xy_ai@Lenovo:~/系统工具开发基础2$ cd q07
(base) xy_ai@Lenovo:~/系统工具开发基础2/q07$ python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
(base) xy_ai@Lenovo:~/系统工具开发基础2/q07$
```

图 2.20: 运行缺陷代码得到错误

3. 在可疑代码行设置断点并启动调试

在下面这一行的行号左侧单击，设置红色断点：

```
result.append(right[i])
```

随后在编辑器的运行菜单中选择 Python Debugger: Debug Python File，启动当前文件的调试。

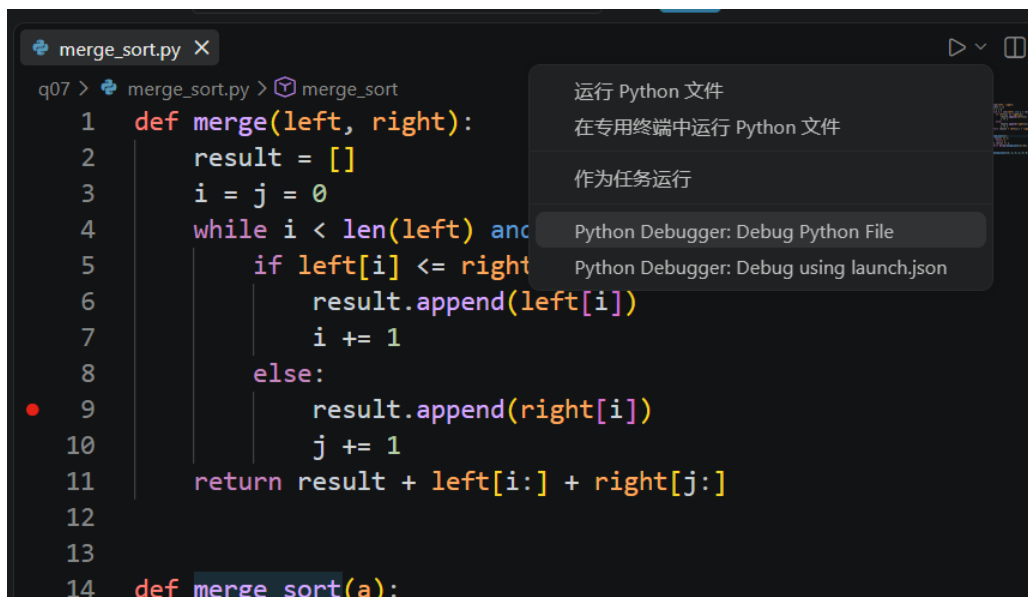


图 2.21

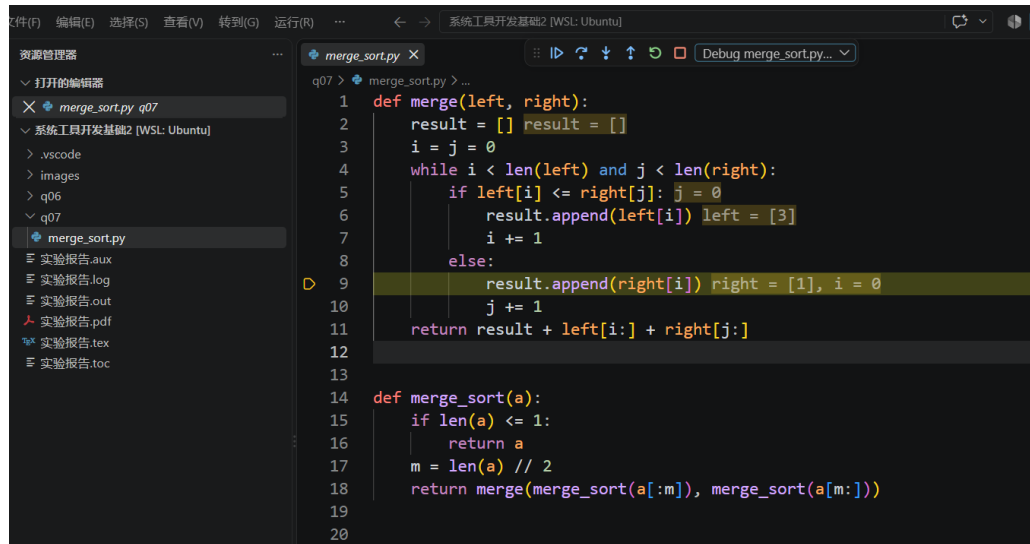


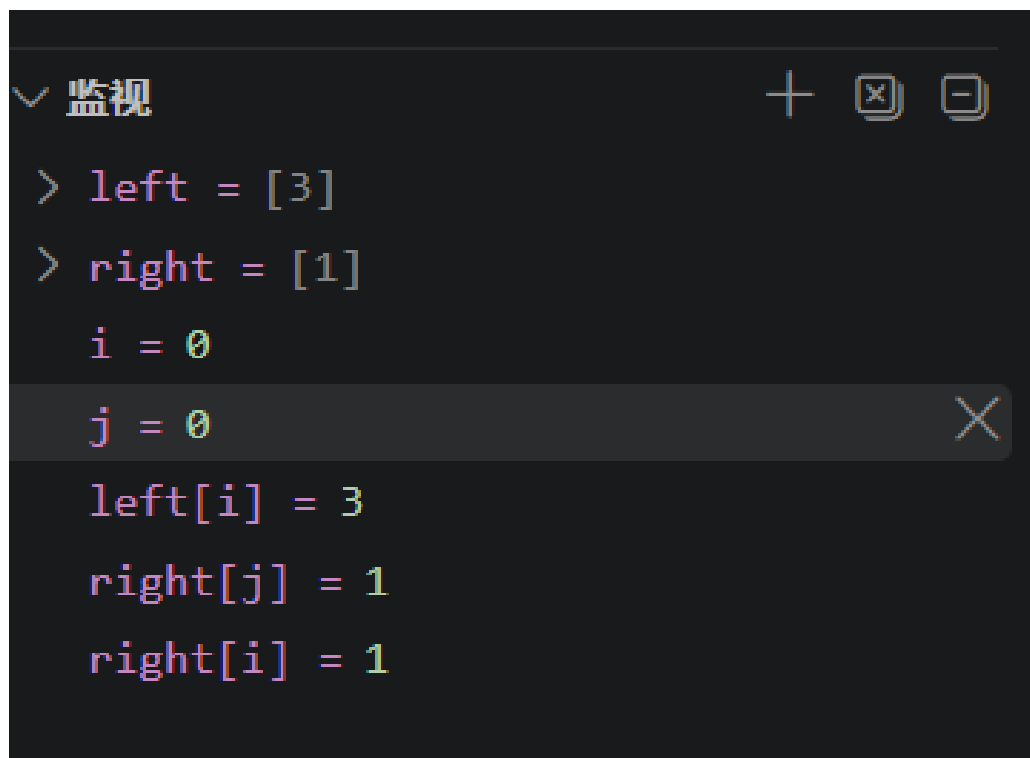
图 2.22: 修改

4. 添加监视表达式并观察多次断点状态

在调试面板的“监视”区域中，加入以下表达式：

```
left
right
i
j
left[i]
right[j]
right[i]
```

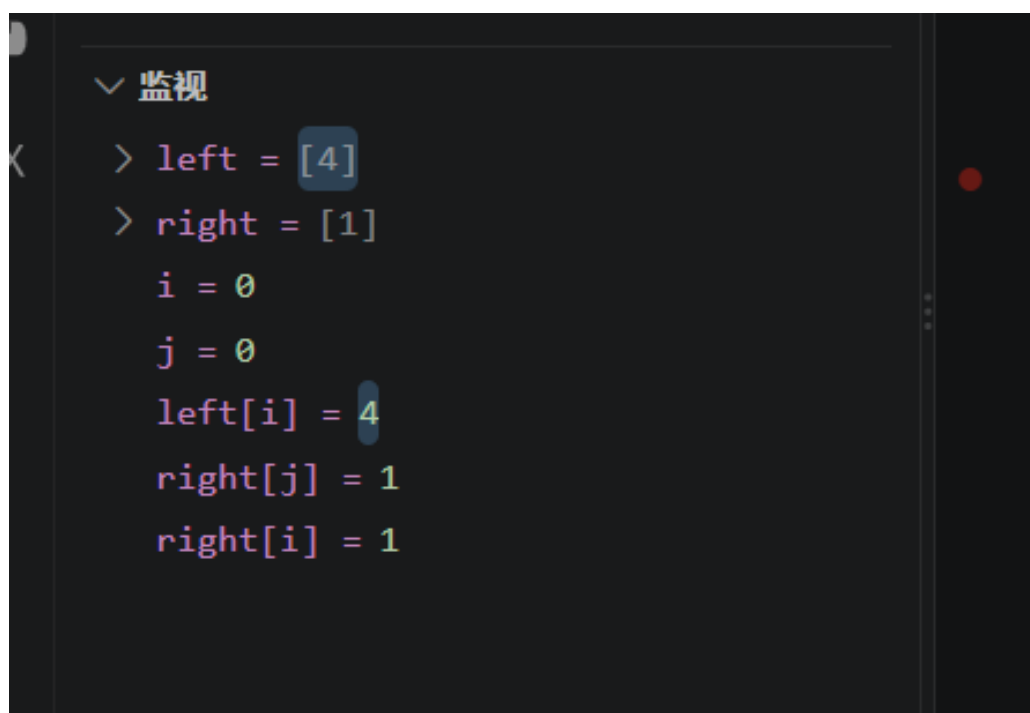
前两次在断点处暂停时，`i` 和 `j` 都等于 0。例如子列表为 `left=[3]`、`right=[1]`，以及 `left=[4]`、`right=[1]` 时，`right[i]` 和 `right[j]` 暂时指向同一个元素，所以缺陷还不明显。



The screenshot shows a debugger's '监视' (Watch) window. It contains a list of variables being monitored: `left`, `right`, `i`, and `j`. The values are: `left = [3]`, `right = [1]`, `i = 0`, and `j = 0`. Below the variable list, the memory addresses and values of these variables are displayed: `left[i] = 3`, `right[j] = 1`, and `right[i] = 1`. The window has a title bar with a minus sign, a close button, and a maximize button.

```
监视  
> left = [3]  
> right = [1]  
i = 0  
j = 0  
left[i] = 3  
right[j] = 1  
right[i] = 1
```

图 2.23: 第一次断点处观察 left、right、i、j 和相应元素



The screenshot shows the same debugger's '监视' (Watch) window after execution. The values have changed: `left = [4]`, `right = [1]`, `i = 0`, and `j = 0`. The memory addresses and values are now: `left[i] = 4`, `right[j] = 1`, and `right[i] = 1`. The window has a title bar with a minus sign, a close button, and a maximize button.

```
监视  
> left = [4]  
> right = [1]  
i = 0  
j = 0  
left[i] = 4  
right[j] = 1  
right[i] = 1
```

图 2.24: 继续执行后观察另一组子列表中的索引状态

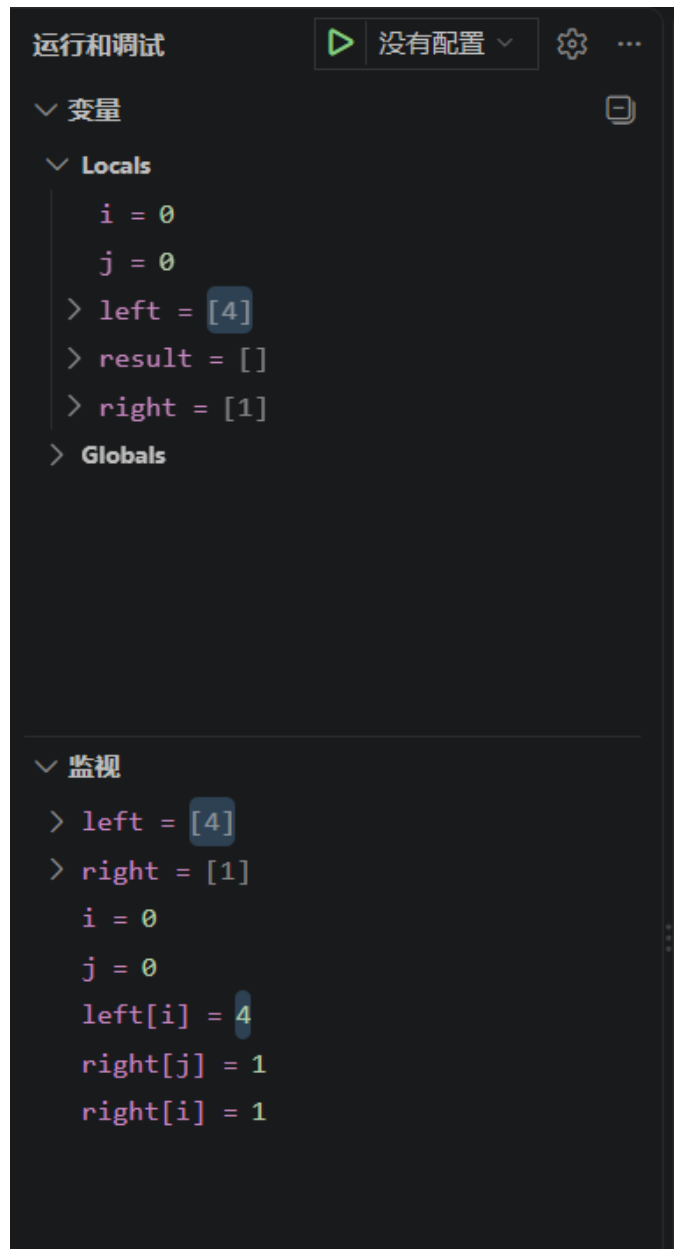


图 2.25: 调试面板同时显示局部变量和监视表达式

继续运行到关键断点时，调试器显示：

```
left = [1, 3]
right = [1, 4]
i = 1
j = 0
left[i] = 3
right[j] = 1
right[i] = 4
```

当前比较为 $3 > 1$ ，程序应从右侧列表的当前位置 `j=0` 取出 `right[j]=1`。但

是缺陷代码按照左侧索引 `i=1` 读取 `right[i]=4`，所以把错误的元素加入了结果。这个运行状态直接定位了缺陷原因。

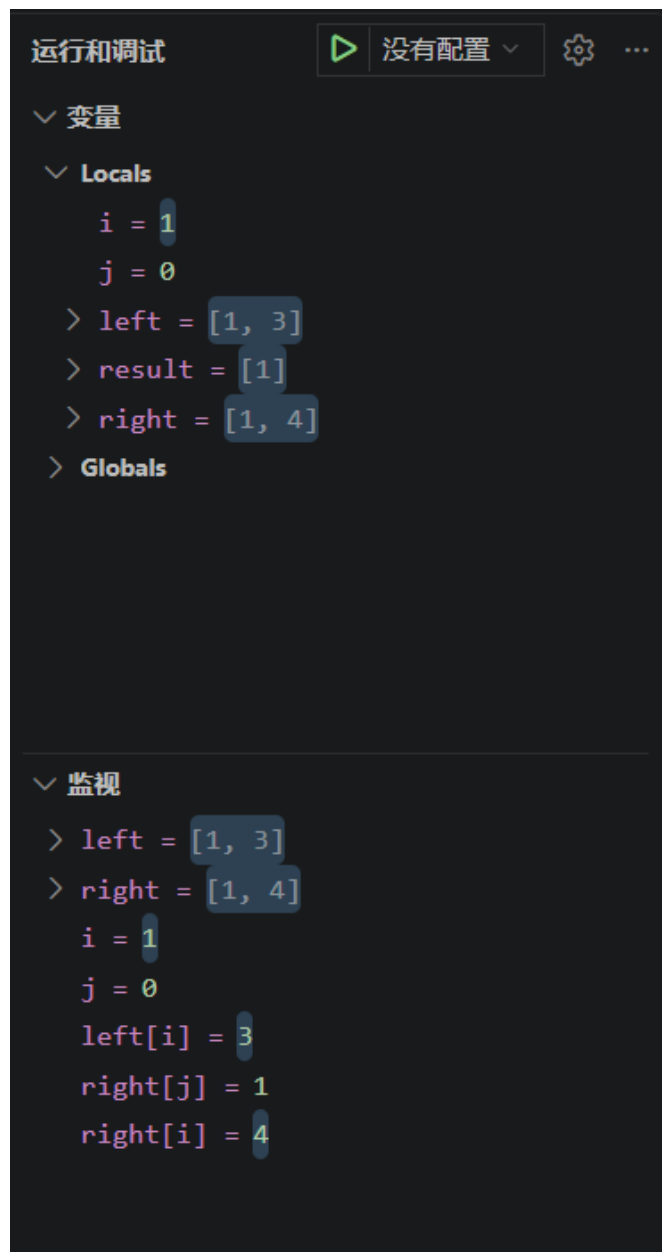


图 2.26: 关键断点状态: `i` 为 1、`j` 为 0, `right[j]` 与 `right[i]` 的值不同

5. 完成最小修复并重新运行

停止调试后，修改产生错误的这一行：

```
# 修改前
result.append(right[i])

# 修改后
result.append(right[j])
```


这是最小修复，因为程序整体的递归拆分、比较、索引递增和剩余元素拼接逻辑均未改变，也没有使用题目禁止的 `sorted`。修改后的关键代码如下图所示。

```
merge_sort.py X
q07 > merge_sort.py > merge_sort
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i])
7             i += 1
8         else:
9             result.append(right[j])
10            j += 1
11    return result + left[i:] + right[j:]
12
```

图 2.27: 把错误的 `right[i]` 最小修改为 `right[j]`

再次执行：

```
python merge_sort.py
```

此时终端输出：

```
[1, 1, 2, 3, 4, 5, 6, 9]
```

结果已经按升序排列，并且两个 1 都得到保留，说明缺陷修复有效。

```
(base) xy_ai@Lenovo:~/系统工具开发基础2$ cd q07
(base) xy_ai@Lenovo:~/系统工具开发基础2/q07$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
(base) xy_ai@Lenovo:~/系统工具开发基础2/q07$
```

图 2.28: 最小修复后得到正确的升序结果

6. 添加两个 Pytest 测试

为了把正确行为固定为可重复执行的检查，我创建 `test_merge_sort.py`，内容如下：

```
from merge_sort import merge_sort
```

```
def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [
        1,
        1,
        2,
        3,
        4,
        5,
        6,
        9,
    ]

def test_duplicate_elements():
    assert merge_sort([4, 2, 4, 1, 2]) == [1, 2, 2, 4, 4]
```

第一个测试覆盖题目给定输入，确认最终顺序准确；第二个测试专门使用包含重复元素的列表，检查归并过程不会丢失或额外产生重复值。这两个测试共同针对本次缺陷可能造成的主要错误。

7. 运行最终测试和静态检查

```
pytest -q
ruff check .
python merge_sort.py
```

Pytest 显示 2 passed in 0.01s，说明给定输入和重复元素两个测试均通过；Ruff 显示 All checks passed!，说明源代码通过静态检查；程序再次输出正确的 [1, 1, 2, 3, 4, 5, 6, 9]。三项结果从测试、代码质量和实际运行三个方面验证了修复。

```
● (base) xy_ai@Lenovo:~/系统工具开发基础2/q07$ pytest -q
..
2 passed in 0.01s
● (base) xy_ai@Lenovo:~/系统工具开发基础2/q07$ ruff check .
All checks passed!
● (base) xy_ai@Lenovo:~/系统工具开发基础2/q07$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
○ (base) xy_ai@Lenovo:~/系统工具开发基础2/q07$
```

图 2.29

2.4 第 8 题先测量再优化慢速词频程序

2.4.1 实验操作过程

1. 创建 q08 和可重复的输入生成程序

VS Code 终端中创建并进入实验目录：

```
cd "/home/xy_ai/系统工具开发基础2"
mkdir -p q08
cd q08
```

创建 generate_words.py：

```
import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]

with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
```

random.seed(20260831) 固定随机数生成器的初始状态，使每次执行都生成相同的单词序列。候选词表由 w0 至 w999 组成，再从中选择 30000 次写入文件。固定输入可以避免优化前后因为输入不同而影响计时结果。

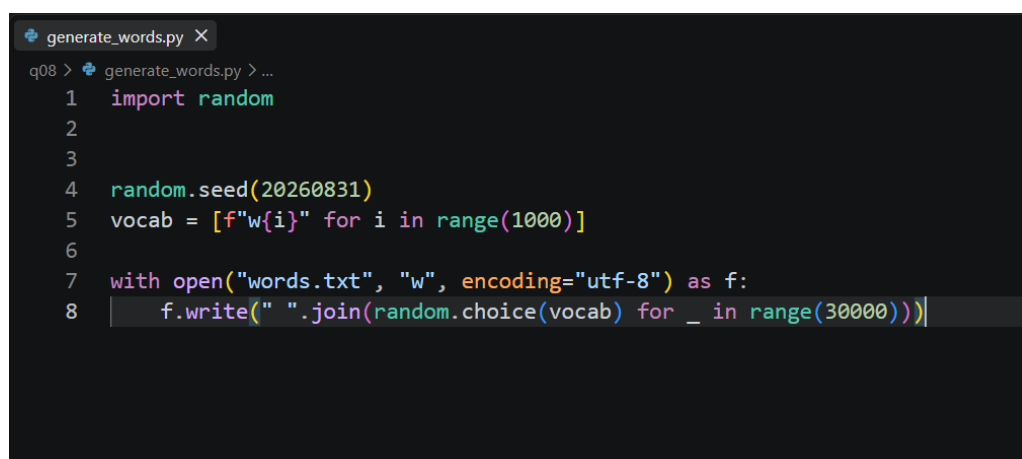


图 2.30: 使用固定随机种子生成包含 30000 个单词的 words.txt

2. 保存故意低效的去重程序

创建 wordfreq.py：

```

words = open("words.txt", encoding="utf-8").read().split()
unique = []

for word in words:
    if word not in unique:
        unique.append(word)

print("count=", len(unique))

```

程序读取并拆分全部单词，再使用列表 `unique` 保存第一次遇到的单词。对于每一个输入单词，`word not in unique` 都可能从列表开头逐个比较。随着 `unique` 变长，重复查找的比较次数不断增加，因此这是潜在的性能瓶颈。

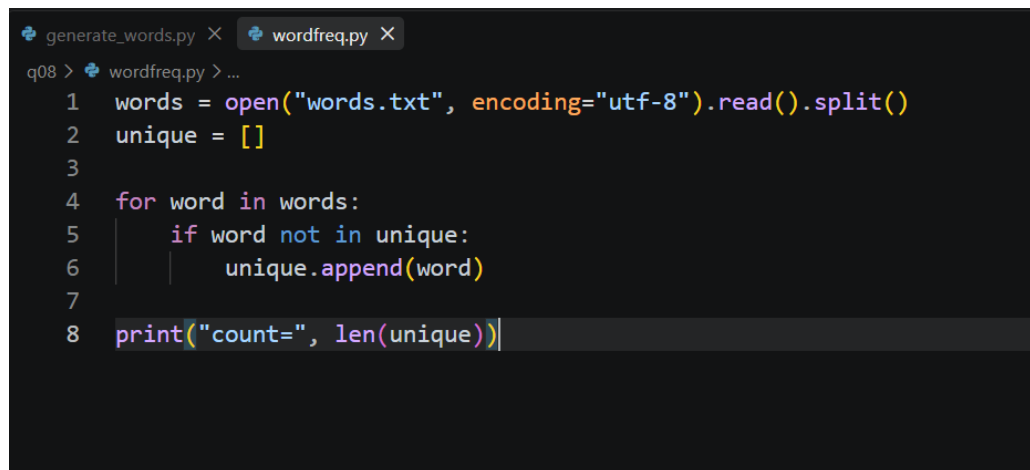


图 2.31

3. 生成并验证相同输入

```

python generate_words.py
ls -lh words.txt
wc -w words.txt
python wordfreq.py

```

`words.txt` 的文件大小约为 144K，`wc -w` 显示 30000，证明输入数量正确。运行原程序输出 `count= 1000`。

4. 使用 `time` 测量优化前的两次耗时

在相同输入上连续执行两次：

```

time python wordfreq.py
time python wordfreq.py

```

两次运行都输出 `count= 1000`，`real` 时间分别为 0.149 秒和 0.174 秒。`real` 表示从命令开始到结束实际经过的墙上时间，包含程序运行以及少量系统调度开销，是本题比较前后速度所采用的数据。

```
(base) xy_ai@Lenovo:~/系统工具开发基础2$ cd q08
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ python generate_words.py
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ ls -lh words.txt
-rw-r--r-- 1 xy_ai xy_ai 144K Sep  7 10:14 words.txt
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ wc -w words.txt
30000 words.txt
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ python wordfreq.py
count= 1000
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ time python wordfreq.py
count= 1000

real    0m0.149s
user    0m0.138s
sys     0m0.008s
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ time python wordfreq.py
count= 1000

real    0m0.174s
user    0m0.165s
sys     0m0.008s
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$
```

图 2.32: 生成 30000 个单词并记录优化前两次 `real` 耗时

5. 使用 CProfile 定位主要耗时位置

```
python -m cProfile -s cumulative wordfreq.py
```

参数 `-s cumulative` 使结果按照累计耗时排序。分析结果显示程序共发生 1012 次函数调用，总时间为 0.156 秒；`wordfreq.py:1(<module>)` 的累计耗时同样为 0.156 秒、自身耗时为 0.154 秒，而读取文件和 `split` 只占约 0.001 秒。

CProfile 把直接写在文件顶层的循环统计到 `<module>` 中。结合源码可知，主要耗时不是文件读取，而是循环中反复执行的列表成员检查 `word not in unique`。列表成员查找需要顺序比较，处理 30000 个单词时会产生大量重复操作。

```
(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.156 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000   0.000   0.156    0.156 {built-in method builtins.exec}
1      0.154   0.154   0.156    0.156 wordfreq.py:1(<module>)
1      0.001   0.001   0.001    0.001 {method 'split' of 'str' objects}
1      0.000   0.000   0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1000    0.000   0.000   0.000    0.000 {method 'append' of 'list' objects}
1      0.000   0.000   0.000    0.000 <frozen codecs>:319(decode)
1      0.000   0.000   0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000   0.000   0.000    0.000 {built-in method builtins.print}
1      0.000   0.000   0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000   0.000   0.000    0.000 {built-in method _io.open}
1      0.000   0.000   0.000    0.000 {built-in method builtins.len}
1      0.000   0.000   0.000    0.000 <frozen codecs>:309(__init__)
1      0.000   0.000   0.000    0.000 <frozen codecs>:260(__init__)

(base) xy_ai@Lenovo:~/系统工具开发基础2/q08$
```

图 2.33: CProfile 显示优化前主要时间

6. 使用集合完成优化

列表、循环和成员检查替换为集合构造，优化后的 wordfreq.py 为：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)

print("count=", len(unique))
```

集合使用哈希结构，创建集合时会自动去除重复元素，平均情况下的查找与插入成本接近常数时间。因此，一次 `set(words)` 就可以完成去重，避免对不断增长的列表反复线性扫描。

程序仍然通过 `len(unique)` 根据输入实际计算数量，没有把结果 1000 写死，也没有改变最终功能。

```
generate_words.py X wordfreq.py X
q08 > wordfreq.py > ...
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set(words)
3
4 print("count=", len(unique))
```

图 2.34

7. 使用相同输入测量优化后的两次耗时

保存后先运行程序确认仍输出 `count= 1000`，执行两次计时：

```
python wordfreq.py
time python wordfreq.py
time python wordfreq.py
```

两次优化后的 `real` 时间分别为 0.026 秒和 0.024 秒。使用的仍然是之前生成并经过 `wc -w` 验证的同一个 `words.txt`，因此可以直接与优化前结果比较。

我还再次运行 CProfile。优化后程序只有 12 次函数调用，总时间为 0.002 秒，`wordfreq.py` 顶层累计时间也降至 0.002 秒。该结果进一步说明原来的列表查重循环已经被消除。

```
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ python wordfreq.py
count= 1000
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ time python wordfreq.py
count= 1000

real    0m0.026s
user    0m0.020s
sys     0m0.005s
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ time python wordfreq.py
count= 1000

real    0m0.024s
user    0m0.019s
sys     0m0.006s
• (base) xy_ai@Lenovo:~/系统工具开发基础2/q08$ python -m cProfile -s cumulative wordfreq.py
count= 1000
      12 function calls in 0.002 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1    0.000    0.000    0.002    0.002 {built-in method builtins.exec}
   1    0.001    0.001    0.002    0.002 wordfreq.py:1(<module>)
   1    0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
   1    0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
   1    0.000    0.000    0.000    0.000 <frozen codecs>:319(decode)
   1    0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
   1    0.000    0.000    0.000    0.000 {built-in method builtins.print}
   1    0.000    0.000    0.000    0.000 {built-in method _io.open}
   1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
   1    0.000    0.000    0.000    0.000 <frozen codecs>:309(__init__)
   1    0.000    0.000    0.000    0.000 {built-in method builtins.len}
   1    0.000    0.000    0.000    0.000 <frozen codecs>:260(__init__)

• (base) xy_ai@Lenovo:~/系统工具开发基础2/q08$
```

图 2.35: 优化后两次计时和 CProfile 结果

2.4.2 中位数与加速比计算

本题每个版本各测量两次。两个数值的中位数取排序后中间两个数的平均值，因此优化前中位耗时为：

$$\frac{0.149 + 0.174}{2} = 0.1615 \text{ 秒}$$

优化后中位耗时为：

$$\frac{0.026 + 0.024}{2} = 0.025 \text{ 秒}$$

加速比使用优化前中位数除以优化后中位数：

$$\frac{0.1615}{0.025} = 6.46$$

因此，在相同输入和相同输出条件下，优化后程序的中位耗时由 0.1615 秒下降到 0.025 秒，约获得 6.46 倍加速。CProfile 的 0.156 秒与 0.002 秒用于分析程序内部耗时，不与 `time` 的墙上时间混合计算正式加速比。

Chapter 3

课后练习部分

3.1 课后第 1 题

问题如下：

理解命令行中的选项终止符 `--`。使用 `touch -- -myfile` 创建一个以连字符开头的文件，然后在不使用 `--` 的情况下将其删除，并解释两种写法产生不同结果的原因。

3.1.1 实验操作过程

1. 创建并进入课后第 1 题目录

为了把课后题与课上完成的 q05 至 q08 分开保存，我在第二次作业目录下建立 `homework/01`：

```
cd assignment-2
mkdir -p homework/01
cd homework/01
```

2. 使用选项终止符创建特殊文件名

```
touch -- -myfile
ls
```

`touch` 用于创建空文件；其中 `--` 结束选项解析，使后面的 `-myfile` 被解释为文件名。随后 `ls` 输出 `-myfile`，证明这个以连字符开头的文件已经创建成功。

3. 直接删除并观察失败结果

尝试直接执行：

```
rm -myfile
```

终端报告 `rm: invalid option -- 'm'`，并提示可以使用 `rm ./-myfile`。这是因为 `rm` 看到 `-myfile` 以连字符开头后，把其中的内容当成命令选项解析；`-m` 不是 `rm` 支持的选项，因此命令失败，文件没有被删除。这个失败现象正好验证了题目所说明的选项解析规则。

4. 不使用 `--` 正确删除文件

按照题目要求，我没有在删除命令中使用 `--`，而是执行：

```
rm ./-myfile  
ls
```

`./` 表示当前目录，因此 `./-myfile` 和 `-myfile` 指向同一个文件。由于传给 `rm` 的参数现在以 `.` 开头，命令将其识别为普通路径并成功删除。最后一次 `ls` 没有任何输出，证明目录中已不存在该文件。

```
(base) xy_ai@Lenovo:~/系统工具开发基础$ cd assignment-2  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ mkdir -p homework/01  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/01  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ touch -- -myfile  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ ls  
-myfile  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ rm -myfile  
rm: invalid option -- 'm'  
Try 'rm ./-myfile' to remove the file '-myfile'.  
Try 'rm --help' for more information.  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ rm ./-myfile  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ ls  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ |
```

图 3.1: 课后第 1 题：创建以连字符开头的文件，观察直接删除失败，并使用相对路径成功删除

3.2 课后第 2 题

问题如下：

使用 `diff` 和 `Bash` 的进程替换比较 `printenv` 与 `export` 的排序输出，执行 `diff <(printenv | sort) <(export | sort)`，观察两者的差异并解释产生差异的原因。

3.2.1 实验操作过程

1. 创建并进入课后第 2 题目录

返回第二次作业的 homework 目录，建立并进入 02：

```
cd
cd 系统工具开发基础/assignment-2/homework
mkdir 02
cd 02
```

2. 查看并排序 printenv 输出

```
printenv | sort | head
```

printenv 输出当前进程能够取得的环境变量，格式为 NAME=value。sort 按文本顺序排序，head 只显示前十行，便于观察。终端显示了 CONDA_DEFAULT_ENV=base、HOME=/home/xy_ai 等内容，说明当前实验在 Conda 的 base 环境和用户 xy_ai 的 Bash 会话中进行。

3. 查看并排序 export 输出

```
export | sort | head
```

不带参数的 export 会列出当前 Shell 中被标记为导出的变量。Bash 使用可重新作为 Shell 声明读取的格式输出，例如 declare -x HOME="/home/xy_ai"。与 printenv 相比，变量名称和值大多对应，但每一行增加了 declare -x，字符串值也使用引号表示。

```
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/01$ cd
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework$ cd 系统工具开发基础/assignment-2/homework
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework$ mkdir 02
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework$ cd 02
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ printenv | sort | head
CONDA_DEFAULT_ENV=base
CONDA_EXE=/home/xy_ai/miniconda3/bin/conda
CONDA_PREFIX=/home/xy_ai/miniconda3
CONDA_PROMPT_MODIFIER=(base)
CONDA_PYTHON_EXE=/home/xy_ai/miniconda3/bin/python
CONDA_SHLVL=1
DISPLAY=:0
HOME=/home/xy_ai
HOSTTYPE=x86_64
HTTPS_PROXY=http://127.0.0.1:1088
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ export | sort | head
declare -x CONDA_DEFAULT_ENV="base"
declare -x CONDA_EXE="/home/xy_ai/miniconda3/bin/conda"
declare -x CONDA_PREFIX="/home/xy_ai/miniconda3"
declare -x CONDA_PROMPT_MODIFIER="(base)"
declare -x CONDA_PYTHON_EXE="/home/xy_ai/miniconda3/bin/python"
declare -x CONDA_SHLVL="1"
declare -x DISPLAY=":0"
declare -x HOME="/home/xy_ai"
declare -x HOSTTYPE="x86_64"
declare -x HTTPS_PROXY="http://127.0.0.1:1088"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ diff <(printenv | sort) <(export | sort)
1,43c1,42
< CONDA_DEFAULT_ENV=base
< CONDA_EXE=/home/xy_ai/miniconda3/bin/conda
```

图 3.2: 课后第 2 题：分别查看 printenv 与 export 的排序结果，并使用进程替换交给 diff 比较

4. 使用 diff 和进程替换比较输出

```
diff <(printenv | sort) <(export | sort)
```

第一处 `<(printenv | sort)` 把排序后的环境变量输出作为第一个临时输入，第二处 `<(export | sort)` 把排序后的导出声明作为第二个临时输入。`diff` 输出开头的 `1,43c1,42` 表示第一个输入的第 1 至 43 行需要改变为第二个输入的第 1 至 42 行。以 `<` 开头的行来自第一个输入，以 `>` 开头的行来自第二个输入。截图中，第一个输入包含 `CONDA_DEFAULT_ENV=base`，第二个输入的对应内容则是 `declare -x CONDA_DEFAULT_ENV="base"`。因此，即使变量表达的是同一项信息，两种命令使用的文本格式不同，`diff` 仍会把它们判断为不同的行。

5. 检查退出状态并观察临时路径

在 `diff` 结束后，执行：

```
echo $?  
echo <(printenv | sort)
```

`echo $?` 输出 1。对于 `diff` 而言，状态 0 表示两个输入相同，状态 1 表示发现差异，大于 1 才表示比较过程中发生错误。因此，本次的 1 证明比较正常完成并确实发现不同，不能把它理解为命令运行失败。

第二条命令输出 `/dev/fd/63`，直接展示了本次 Bash 为进程替换提供的文件描述符路径。该路径只在相关命令执行期间有效，不是在 `homework/02` 中永久保存的普通文件。

```
declare -x XDG_RUNTIME_DIR="/mnt/wslg/runtime-dir"  
> declare -x _CE_CONDA=""  
> declare -x _CE_M=""  
> declare -x http_proxy="http://127.0.0.1:1088"  
> declare -x https_proxy="http://127.0.0.1:1088"  
> declare -x no_proxy="192.168.*,172.31.*,172.30.*,172.29.*,172.28.*,172.27.*,172.26.*,172.  
.*,172.17.*,172.16.*,10.*,127.*,localhost,<local>"  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ echo $?  
1  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ echo <(printenv | sort)  
/dev/fd/63  
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ |
```

图 3.3: 课后第 2 题：diff 返回状态 1，并显示进程替换产生的 `/dev/fd/63` 路径

3.3 课后第 3 题

问题如下：

对一个很少失败的随机命令编写 Bash 调试脚本，不断运行该命令直到出现失败；把失败时的标准输出和标准错误分别保存到文件中并打印结果，同时报告经过多少次运行才失败。

3.3.1 实验操作过程

1. 创建并进入课后第 3 题目录

```
cd "/home/xy_ai/系统工具开发基础/assignment-2"
mkdir -p homework/03
cd homework/03
```

2. 编写很少失败的目标脚本

rare_failure.sh 内容为：

```
#!/usr/bin/env bash

n=$((RANDOM % 100))

if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    echo "The error was using magic numbers" >&2
    exit 1
fi

echo "Everything went according to plan"
```

Bash 的 RANDOM 每次产生一个随机整数，对 100 取余后得到 0 至 99 的数。只有当结果等于 42 时程序才进入失败分支，因此单次运行失败的概率约为 1/100。失败分支把第一句话写到标准输出，把第二句话通过 >&2 写到标准错误，并使用 exit 1 返回失败状态；其他情况下程序输出成功信息并以状态 0 结束。

3. 编写持续运行并捕获失败的脚本

创建 run_until_fail.sh：

```
#!/usr/bin/env bash

stdout_file="stdout.log"
stderr_file="stderr.log"
attempts=0

: > "$stdout_file"
: > "$stderr_file"

while true; do
```

```
    attempts=$((attempts + 1))

    ./rare_failure.sh > "$stdout_file" 2> "$stderr_file"
    status=$?

    if ((status != 0)); then
        break
    fi
done

echo "Failed after $attempts attempts."
echo "Exit status: $status"

echo "--- Standard output ---"
cat "$stdout_file"

echo "--- Standard error ---"
cat "$stderr_file"
```

变量 `attempts` 从 0 开始，每轮循环先加 1。目标脚本的标准输出和标准错误分别覆盖写入两个日志文件，然后立即用 `status=$?` 保存返回码。如果返回码不为 0，`break` 终止无限循环；否则继续下一轮。退出循环后，脚本打印运行次数、返回码以及两个日志文件的内容。

```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/02$ cd "/home/x
y_ai/系统工具开发基础/assignment-2"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ mkdir -p homework/03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/03
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ nano rare_fa
ilure.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ nano run_un
til_fail.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ cat rare_fa
ilure.sh
#!/usr/bin/env bash

n=$((RANDOM % 100))

if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    echo "The error was using magic numbers" >&2
    exit 1
fi

echo "Everything went according to plan"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ cat run_un
til_fail.sh
#!/usr/bin/env bash

stdout_file="stdout.log"
stderr_file="stderr.log"
attempts=0

: > "$stdout_file"
: > "$stderr_file"

while true; do
    attempts=$((attempts + 1))

    ./rare_failure.sh > "$stdout_file" 2> "$stderr_file"

```

图 3.4: 课后第 3 题: 创建随机失败脚本和持续运行至失败的调试脚本

4. 检查语法并设置执行权限

```

bash -n rare_failure.sh
bash -n run_until_fail.sh
echo $?
chmod +x rare_failure.sh run_until_fail.sh

```

`bash -n` 只检查脚本语法而不执行其中的命令。`echo $?` 输出 0, 说明两个脚本通过语法检查。随后使用 `chmod +x` 为它们增加执行权限, 保证可以通过 `./` 脚本名直接运行。

5. 运行至失败并检查保存结果

```
./run_until_fail.sh
```

本次实验在第 140 次运行时遇到随机失败, 输出 `Failed after 140 attempts.`, 返回状态为 1。脚本随后打印标准输出中的 `Something went wrong`, 以及标准错误中的 `The error was using magic numbers`。

单独验证日志:

```
cat stdout.log
cat stderr.log
ls -l stdout.log stderr.log
```

stdout.log 只包含普通故障提示，stderr.log 只包含错误原因，两种输出没有混合。ls -l 显示两个日志文件均已生成，其中 stdout.log 为 21 字节，stderr.log 为 34 字节。

```
status=$?
if ((status != 0)); then
    break
fi
done

echo "Failed after $attempts attempts."
echo "Exit status: $status"

echo "--- Standard output ---"
cat "$stdout_file"

echo "--- Standard error ---"
cat "$stderr_file"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ bash -n rare_failure.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ bash -n run_until_fail.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ echo $?
0
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ chmod +x rare_failure.sh run_until_fail.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ ./run_until_fail.sh
Failed after 140 attempts.
Exit status: 1
--- Standard output ---
Something went wrong
--- Standard error ---
The error was using magic numbers
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ cat stdout.log
Something went wrong
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ cat stderr.log
The error was using magic numbers
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ ls -l stdout.log stderr.log
-rw-r--r-- 1 xy_ai xy_ai 34 Sep  8 22:21 stderr.log
-rw-r--r-- 1 xy_ai xy_ai 21 Sep  8 22:21 stdout.log
```

图 3.5

3.4 课后第 4 题

问题如下：

递归找出某个目录中最近修改过的文件，并进一步按照“最近修改时间”列出目录中的所有文件。

3.4.1 实验操作过程

1. 创建第 4 题目录和多层测试结构

```
cd "/home/xy_ai/系统工具开发基础/assignment-2"
mkdir -p homework/04
cd homework/04
mkdir -p testdir/subdir/deeper
```


在不同层级创建三个文件，其中一个文件名包含空格：

```
touch testdir/old.txt
touch "testdir/subdir/middle file.txt"
touch testdir/subdir/deeper/newest.txt
```

2. 设置可重复验证的修改时间

为了避免三个文件的创建时间过于接近，使用 `touch -t` 设置明确的修改时间：

```
touch -t 202609081000 testdir/old.txt
touch -t 202609081100 "testdir/subdir/middle file.txt"
touch -t 202609081200 testdir/subdir/deeper/newest.txt
```

三个时间分别表示 2026 年 9 月 8 日 10 时、11 时和 12 时。这样，正确的从新到旧顺序应当是 `newest.txt`、`middle file.txt`、`old.txt`。我执行：

```
find testdir -type f -printf '%TY-%Tm-%Td %TH:%TM:%TS %p\n'
```

输出显示三个文件及其设定时间，且最深层的 `newest.txt` 也被找到，证明 `find` 已递归遍历子目录。此时输出顺序来自目录遍历过程，并不代表时间顺序。

3. 找出最近修改的单个文件

```
find testdir -type f \
  -printf '%T@ %TY-%Tm-%Td %TH:%TM:%TS %p\n' |
  sort -nr | head -n 1 | cut -d' ' -f2-
```

`%T@` 在每行开头输出适合数值排序的修改时间戳，`sort -nr` 使用数值倒序排列，`head -n 1` 只保留第一行。最后，`cut -d' ' -f2-` 去掉开头的排序键，保留可读时间和完整路径。输出为 12 时修改的 `testdir/subdir/deeper/newest.txt`，与预设结果一致。

4. 按修改时间列出全部文件

去掉只保留第一行的 `head` 后执行：

```
find testdir -type f \
  -printf '%T@ %TY-%Tm-%Td %TH:%TM:%TS %p\n' |
  sort -nr | cut -d' ' -f2-
```

输出依次为 12 时的 `newest.txt`、11 时的 `middle file.txt` 和 10 时的 `old.txt`。文件名中的空格被完整保留，全部文件也按最近修改时间从新到旧排列，完成了题目更一般的要求。

5. 编写可重复使用的通用脚本

为了能够查询任意目录，上述逻辑整理为 `recent_files.sh`：

```
#!/usr/bin/env bash

directory="${1:-.}"

if [[ ! -d "$directory" ]]; then
    echo "Error: '$directory' is not a directory." >&2
    exit 1
fi

records=$(
    find "$directory" -type f \
        -printf '%T@ %TY-%Tm-%Td %TH:%TM:%TS %p\n' |
    sort -nr
)

if [[ -z "$records" ]]; then
    echo "No files found in: $directory"
    exit 0
fi

echo "--- Most recently modified file ---"
printf '%s\n' "$records" |
    head -n 1 |
    cut -d' ' -f2-

echo
echo "--- All files, newest first ---"
printf '%s\n' "$records" |
    cut -d' ' -f2-
```

`directory="${1:-.}"` 表示优先使用第一个命令行参数，没有参数时默认查询当前目录。脚本先验证参数是否为目录，再把递归查找并排序后的结果保存到 `records`。如果没有找到文件则给出提示；否则分别输出最近的一个文件和完整排序列表。

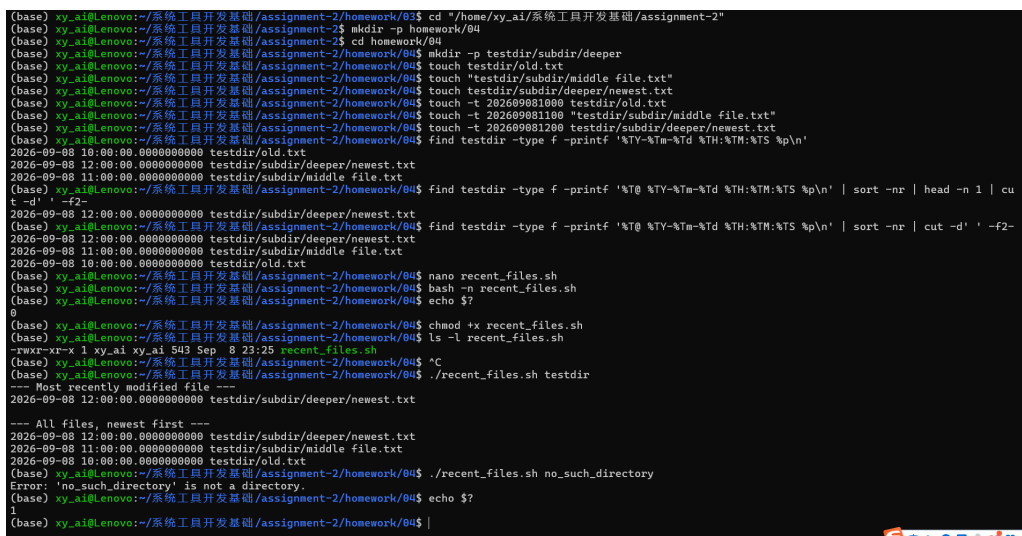
6. 验证脚本和错误处理

```

bash -n recent_files.sh
echo $?
chmod +x recent_files.sh
ls -l recent_files.sh
./recent_files.sh testdir
./recent_files.sh no_such_directory
echo $?

```

语法检查返回 0, `ls -l` 中出现执行权限。查询 `testdir` 时, 脚本正确报告最近文件并列出 12 时、11 时、10 时的全部文件; 传入不存在的 `no_such_directory` 时, 错误信息被写到标准错误, 脚本返回 1, 说明参数检查有效。



```

(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/03$ cd ~/home/xy_ai/系统工具开发基础/assignment-2"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ mkdir -p homework/04
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/04
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ mkdir -p testdir/subdir/deeper
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch testdir/old.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch "testdir/subdir/middle file.txt"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch testdir/subdir/deeper/newest.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch -t 202609081000 testdir/old.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch -t 202609081100 "testdir/subdir/middle file.txt"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ touch -t 202609081200 testdir/subdir/deeper/newest.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ find testdir -type f -printf '%TY-%Tm-%Td %TH:%TS %p\n'
2026-09-08 10:00:00.000000000 testdir/old.txt
2026-09-08 12:00:00.000000000 testdir/subdir/deeper/newest.txt
2026-09-08 11:00:00.000000000 testdir/subdir/middle file.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ find testdir -type f -printf '%T@ %TY-%Tm-%Td %TH:%TS %p\n' | sort -nr | head -n 1 | cut
-t -d' ' -f2-
2026-09-08 12:00:00.000000000 testdir/subdir/deeper/newest.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ find testdir -type f -printf '%T@ %TY-%Tm-%Td %TH:%TS %p\n' | sort -nr | cut -d' ' -f2-
2026-09-08 11:00:00.000000000 testdir/subdir/middle file.txt
2026-09-08 10:00:00.000000000 testdir/old.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ nano recent_files.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ bash -n recent_files.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ echo $?
0
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ chmod +x recent_files.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ ls -l recent_files.sh
-rwxr-xr-x 1 xy_ai xy_ai 549 Sep  9 22:25 recent_files.sh
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ ^C
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ ./recent_files.sh testdir
--- Most recently modified file ---
2026-09-08 12:00:00.000000000 testdir/subdir/deeper/newest.txt
--- All files, newest first ---
2026-09-08 12:00:00.000000000 testdir/subdir/deeper/newest.txt
2026-09-08 11:00:00.000000000 testdir/subdir/middle file.txt
2026-09-08 10:00:00.000000000 testdir/old.txt
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ ./recent_files.sh no_such_directory
Error: 'no_such_directory' is not a directory.
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$ echo $?
1
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/04$

```

图 3.6

3.5 课后第 5 题

问题如下:

创建一个名为 `dc` 的命令别名, 使得把 `cd` 误输入成 `dc` 时仍能正常切换目录。

3.5.1 实验操作过程

1. 创建并进入课后第 5 题目录

```

cd "/home/xy_ai/系统工具开发基础/assignment-2"
mkdir -p homework/05
cd homework/05

```

终端提示符随后显示当前位置为 `assignment-2/homework/05`。

2. 创建并检查 `dc` 别名

当前 Bash 会话中执行：

```
alias dc='cd'
alias dc
```

第一条命令把名称 `dc` 绑定到 Bash 内置命令 `cd`。第二条命令专门查询这个别名，终端输出 `alias dc='cd'`，证明别名已经成功注册。

3. 使用别名完成目录切换

为了验证别名能够像原命令一样接收路径参数：

```
dc ~/系统工具开发基础
```

Bash 先把命令开头的 `dc` 展开为 `cd`，然后保留后面的目录参数，因此实际效果相当于执行 `cd /系统工具开发基础`。执行后，终端提示符从 `assignment-2/homework/05` 变为 `/系统工具开发基础`，证明当前工作目录已成功改变，也说明把 `cd` 错打成 `dc` 时仍能正常工作。

```
(base) xy_ai@Lenovo:~/ssh$ cd "/home/xy_ai/系统工具开发基础/assignment-2"
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ mkdir -p homework/05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/05
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/05$ alias dc='cd'
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/05$ alias dc
alias dc='cd'
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/05$ dc ~/系统工具开发基础
(base) xy_ai@Lenovo:~/系统工具开发基础$ |
```

图 3.7: 课后第 5 题：创建并查询 `dc` 别名，然后使用 `dc` 成功切换工作目录

3.6 课后第 6 题

问题如下：

浏览一份 IDE 扩展列表，从中选择并安装一个看起来对自己的开发工作有用的扩展。

3.6.1 实验操作过程

1. 浏览扩展详情并确认选择

浏览 GitLens 扩展详情并根据 Git 版本控制需求决定安装：



图 3.8

2. 安装扩展并确认 WSL 环境中的状态

GitLens 安装完成，并显示已在 WSL Ubuntu 上启用：



图 3.9

3.7 课后第 7 题

问题如下：

二分查找程序在查找不存在的数字时会陷入死循环。使用 VS Code 调试器定位问题，并进行最小修复。

```

• (base) xy_ai@Lenovo:~/系统工具开发基础$ cd assignment-2/homework/07
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ nano binary_search.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ cat binary_search.py
def binary_search(nums, target):
    low = 0
    high = len(nums) - 1

    while low <= high:
        mid = (low + high) // 2

        if nums[mid] == target:
            return mid

        if nums[mid] < target:
            low = mid
        else:
            high = mid - 1

    return -1

print(binary_search([1, 3, 5, 7, 9], 8))
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ timeout 2s python binary_search.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ echo $?
124

```

图 3.10: 程序超时退出，返回码为 124

调试发现 low=3、high=4、mid=3 重复不变，搜索范围没有缩小。

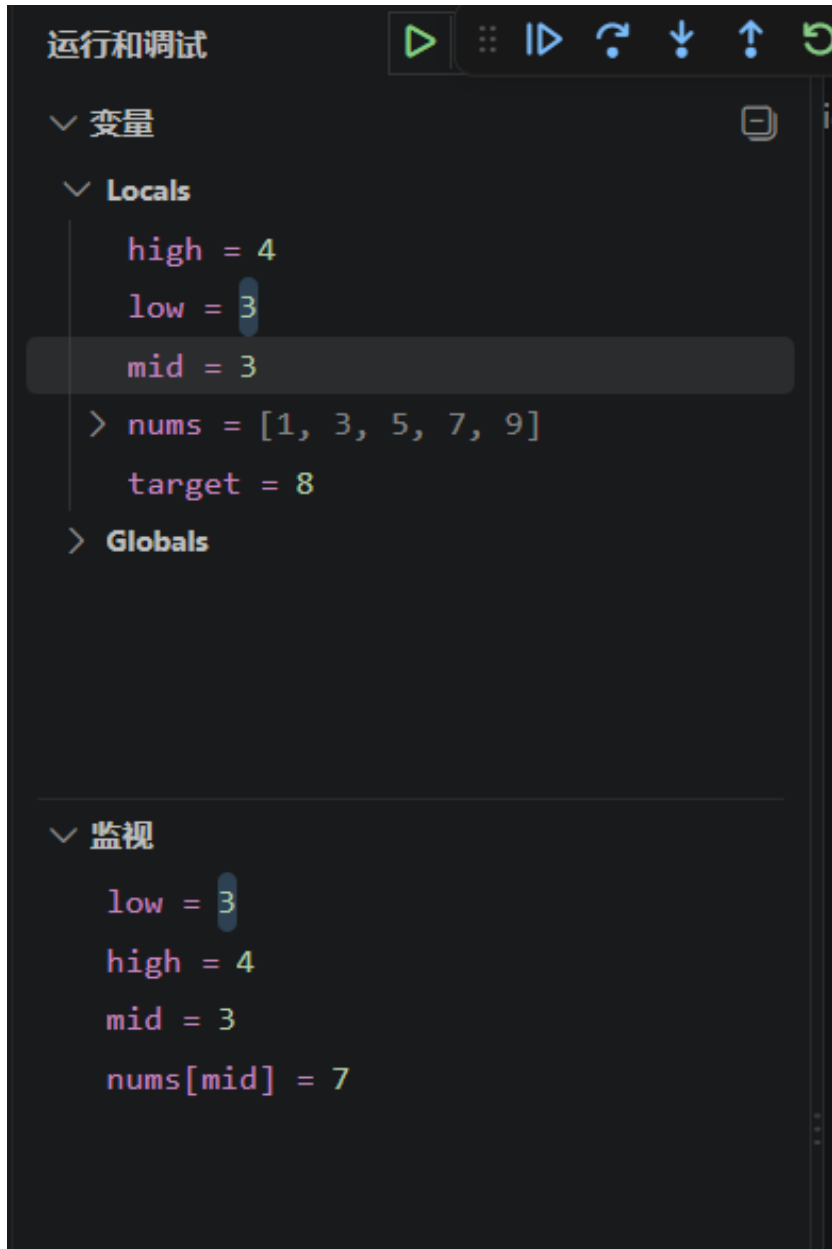


图 3.11: 调试器中重复不变的变量状态

将 `low = mid` 改为 `low = mid + 1` 后，程序输出 `-1`，Ruff 检查通过。

```
(base) xy_ai@Lenovo:~/系统工具开发基础$ /usr/bin/env /usr/bin/python3 /home/xy_ai/.vscode-server/e
ndled/libs/debugpy/adapter/../../debugpy/launcher 46099 -- /home/xy_ai/系统工具开发基础/assignment-
(base) xy_ai@Lenovo:~/系统工具开发基础$ cd assignment-2/homework/07
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ python binary_search.py
-1
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$ ruff check binary_search.py
All checks passed!
(base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/07$
```

图 3.12: 修复后的运行结果与 Ruff 检查结果

3.8 课后第 8 题

问题如下：

删除列表负数的程序会遗漏连续负数。使用 VS Code 调试器定位问题并修复。

程序初次输出 [3, -2, 4, -6]。调试时发现删除元素后，下标递增会跳过新的当前元素。

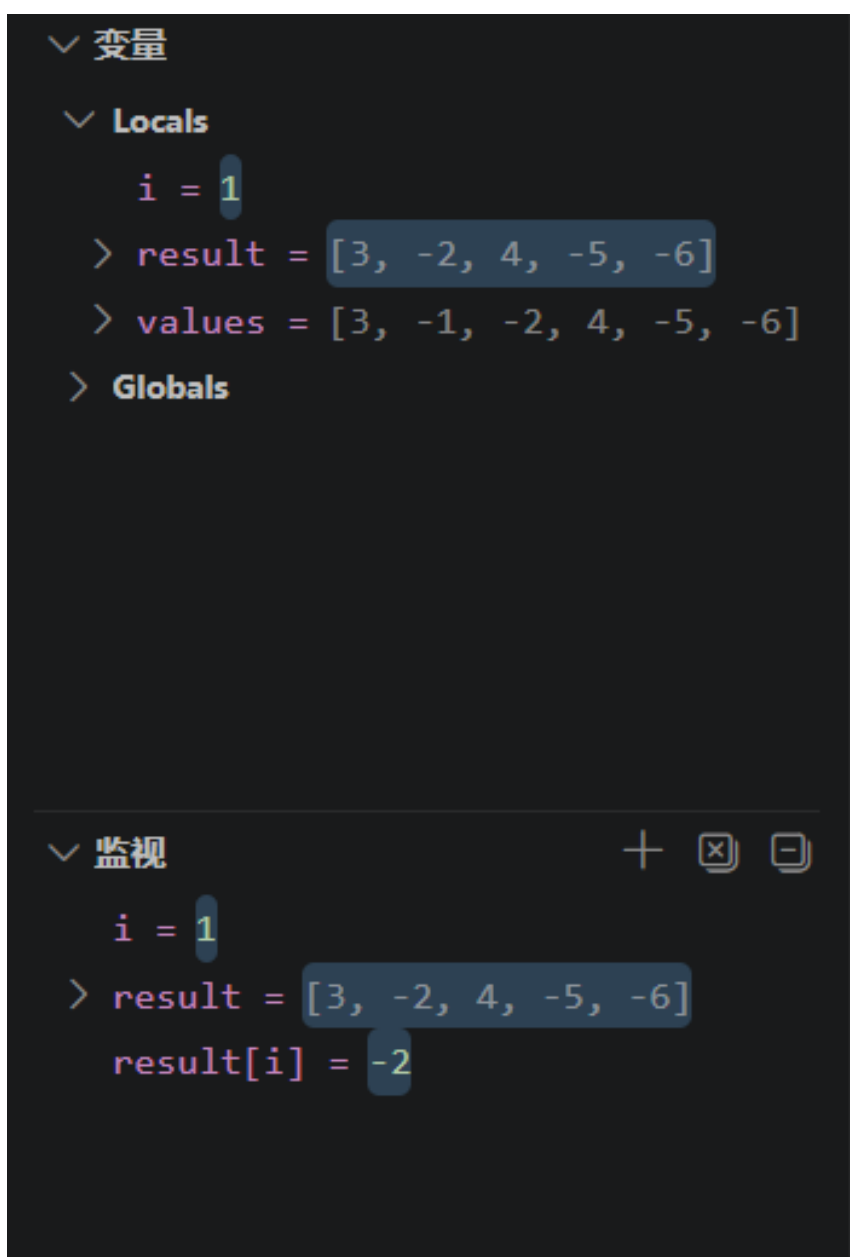


图 3.13: 调试器显示被跳过的负数

只在未删除元素时递增下标，程序正确输出 [3, 4]，Ruff 检查通过。


```
common_items.py U X
assignment-2 > homework > 09 > common_items.py > find_common
1 def find_common(first, second):
2     common = []
3
4     for number in first:
5         if number in second:
6             common.append(number)
7
8     return common
9
10
11 first = list(range(20000))
12 second = set(range(10000, 30000))
13
14 result = find_common(first, second)
15 print(len(result))
16
```

图 3.15: 优化前后运行结果与 cProfile 分析

优化后的代码使用集合进行成员查找，Ruff 检查通过。

```

• (base) xy_ai@Lenovo:~/系统工具开发基础$ cd "/home/xy_ai/系统工具开发基础/assignment-2"
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ mkdir -p homework/09
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/09
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ nano common_items.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ python common_items.py
10000
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ python -m cProfile -s cumulative common_items.py
10000

10006 function calls in 1.205 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    1.205    1.205 {built-in method builtins.exec}
1      0.001    0.001    1.205    1.205 common_items.py:1(<module>)
1      1.203    1.203    1.204    1.204 common_items.py:1(find_common)
10000  0.002    0.000    0.002    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ nano common_items.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ nano common_items.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ python -m cProfile -s cumulative common_items.py
10000

10006 function calls in 0.004 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.004    0.004 {built-in method builtins.exec}
1      0.002    0.002    0.004    0.004 common_items.py:1(<module>)
1      0.001    0.001    0.002    0.002 common_items.py:1(find_common)
10000  0.001    0.000    0.001    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}

• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ ruff check common_items.py
All checks passed!
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/09$ █

```

图 3.16: 使用集合优化查找代码

3.10 课后第 10 题

问题如下：

使用性能分析工具优化循环中的字符串拼接程序。

将循环中的字符串累加改为列表收集后使用 `join`，两次输出长度均为 288890，`cProfile` 显示优化后调用方式改变，`Ruff` 检查通过。

```

• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2$ cd homework/10
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ nano build_text.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ python build_text.py
288890
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ python -m cProfile -s cumulative build_text.py
288890

 6 function calls in 0.021 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1   0.000   0.000   0.021   0.021 {built-in method builtins.exec}
   1   0.000   0.000   0.021   0.021 build_text.py:1(<module>)
   1   0.021   0.021   0.021   0.021 build_text.py:1(build_text)
   1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
   1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
   1   0.000   0.000   0.000   0.000 {built-in method builtins.len}

• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ nano build_text.py
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ python build_text.py
288890
• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ python -m cProfile -s cumulative build_text.py
288890

50007 function calls in 0.018 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1   0.000   0.000   0.018   0.018 {built-in method builtins.exec}
   1   0.000   0.000   0.018   0.018 build_text.py:1(<module>)
   1   0.013   0.013   0.017   0.017 build_text.py:1(build_text)
 50000   0.004   0.000   0.004   0.000 {method 'append' of 'list' objects}
   1   0.000   0.000   0.000   0.000 {method 'join' of 'str' objects}
   1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
   1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
   1   0.000   0.000   0.000   0.000 {built-in method builtins.len}

• (base) xy_ai@Lenovo:~/系统工具开发基础/assignment-2/homework/10$ ruff check build_text.py
All checks passed!

```

图 3.17: 优化前后运行结果与 cProfile 分析

```

build_text.py U X
assignment-2 > homework > 10 > build_text.py > build_text

1 def build_text():
2     parts = []
3
4     for number in range(50000):
5         parts.append(str(number) + ",")
6
7     return "".join(parts)
8
9
10 result = build_text()
11 print(len(result))
12

```

图 3.18: 使用列表和 join 优化字符串拼接

Chapter 4

解题感悟

4.1 课上练习

1. 第 5 题中，我以前把 `Ctrl-Z` 当成结束程序，做完后才知道它只是暂停，之后还能用 `bg` 继续运行。
2. 第 6 题让我发现，重命名函数时用编辑器的“重命名符号”比较方便，几个文件里的调用可以一起改。`Ruff` 也能很快找到没有使用的导入。
3. 第 7 题中，第一次停在断点时还看不出问题，继续运行到 `i` 和 `j` 不同时，才发现代码用了错误的下标。
4. 第 8 题让我知道优化前要先计时。把列表去重改成集合后，输出没有变化，但运行快了很多。

4.2 课后练习

1. 课后第 1 题：文件名以连字符开头时容易被当成选项，在前面加 `./` 就能明确表示文件路径。
2. 课后第 2 题：`printenv` 和 `export` 显示的格式不同，进程替换可以让 `diff` 直接比较两条命令的输出。
3. 课后第 3 题：用循环可以自动等到偶尔出现的错误，并把当时的两种输出分别保存下来。
4. 课后第 4 题：`find` 找到文件后，还要按修改时间排序才能确定哪

个文件最新。

5. 课后第 5 题：别名能改正经常输错的命令，不过当前设置只在这个终端中有效。
6. 课后第 6 题：我选择 GitLens，是因为以后查看 Git 的提交记录时可能会用到。
7. 课后第 7 题：变量一直不变说明二分查找的范围没有继续缩小。
8. 课后第 8 题：删除元素后，后面的元素会向前移动，所以下标不能马上加一。
9. 课后第 9 题：把列表查找改成集合查找后，性能分析显示速度提高很多。
10. 课后第 10 题：字符串较多时，可以先放进列表，最后再用 `join` 拼接。

Chapter 5

上传 GitHub

5.1 编写并更新.gitignore

在提交第二次实验作业前，我继续在课程仓库根目录维护 `.gitignore`，使其同时适用于两次作业。除 LaTeX 编译中间产物、Windows 下载标记和个人 VS Code 配置外，本次还增加了 Python 运行与测试产生的缓存，以及第 8 题中能够由脚本重新生成的词表文件。当前规则如下：

```
# LaTeX 编译中间产物
*.aux
*.fdb_latexmk
*.fls
*.log
*.out
*.synctex.gz
*.xdv
*.toc

# Python 缓存和测试工具缓存
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/

# 第二次实验可重新生成的数据
/assignment-2/q08/words.txt

# Windows 下载标记
```



```
*Zone.Identifier*
```

```
# 个人 IDE 配置
```

```
.vscode/
```

其中, *.out 是 LaTeX 超链接等功能生成的辅助文件; __pycache__/ 和 *.py[cod] 用于忽略 Python 字节码缓存; .pytest_cache/ 与 .ruff_cache/ 分别是 Pytest 和 Ruff 的本地缓存。第 8 题的 q08/words.txt 可由 generate_words.py 重新生成, 因此不必作为源码提交。这样既能保留作业代码、实验报告和截图, 又不会把可再生文件或本机工具状态混入版本记录。

5.2 GitHub 仓库链接

作业仓库地址 (公开, 默认分支为 main):

https://github.com/xiaoyang1088/system_development_tools

仓库用户名为 xiaoyang1088, 仓库名为 system_development_tools。第二次作业保存在 assignment-2/ 目录下, 包含 q05 至 q08、homework/、images/ 和实验报告文件; 更新后的 .gitignore 位于课程仓库根目录, 对整个仓库生效。

5.3 commit 提交记录

本次作业按完成进度进行了多次提交。提交历史中可以看到先后完成第 5 次提交的实验报告、更新 .gitignore 并上传 q05-q08、补充课后作业文件夹和报告截图, 最后提交第二次实验报告。这样的提交过程保留了作业逐步完善的记录, 也便于回溯每次变更。

Commits on Sep 11, 2026	提交第二次实验报告(不含commit截图) xiaoyang1088 committed 1 minute ago	d36aa43	📄	<>
Commits on Sep 9, 2026	添加第二次实验报告截图 xiaoyang1088 committed 2 days ago	8edd1cc	📄	<>
	添加作业文件夹 01 to 10 xiaoyang1088 committed 2 days ago	685e810	📄	<>
Commits on Sep 8, 2026	上传新的.gitignore和第二次课作业文件夹q05-q08 xiaoyang1088 committed 3 days ago	8413363	📄	<>
	为了第二次作业上交整理第一次作业的文件 xiaoyang1088 committed 3 days ago	c4d53b3	📄	<>
Commits on Sep 1, 2026	第5次提交：包含commit记录的实验报告 xiaoyang1088 committed 2 weeks ago	4312e4a	📄	<>
	第4次提交：实验报告 xiaoyang1088 committed 2 weeks ago	abe0213	📄	<>
Commits on Aug 31, 2026	第3次提交：上传课后homework练习 xiaoyang1088 committed 2 weeks ago	be673a4	📄	<>
	第2次提交：完成q01-q04题目代码 xiaoyang1088 committed 2 weeks ago	1c1b7c3	📄	<>

图 5.1: 第二次实验作业的 GitHub commit 提交记录